Drowsiness Detection System Using Python

A PROJECT REPORT

Submitted by

SENTHILKUMAR.P (REG.NO:953720104042)

SHANMUGAVEL.V (REG.NO:953720104043)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

AAA COLLEGE OF ENGINEERING AND TECHNOLOGY, SIVAKASI

ANNA UNIVERSITY:CHENNAI 600025

MAY 2023

ANNA UNIVERSITY : CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report **" Drowsiness Detection System Using Python"**is the bonafide work of **"SENTHILKUMAR.P (Reg.No:953740104042), SHANMUGAVEL.V (Reg.No:953740104043)"** who carried out the project under my supervision.

SIGNATURE
Dr. J. Sutha, M.E. ,Ph.D.
Professor & Head
Computer Science & Engineering
AAA College of Engineering & Technology.,
Sivakasi – 626 005
Virudhunagar District.

SIGNATURE
Mr.J.Vinoth kumar, M.E.,
Assistant Professor
Computer Science & Engineering
AAA College of Engineering & Technology.,
Sivakasi – 626 005
Virudhunagar District.

Submitted for the project viva-voce examination to be held on_____.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

First and foremost we **praise and thank "The Almighty",** the lord of all creations, who by his abundant grace has sustained us and helped us to work on this project successfully.

We really find unique pleasure and immense gratitude in thanking our respected management members**,** who is the backbone of our college.

A deep bouquet of thanks to respected Principal **Dr. M. Sekar M.E.,Ph.D.,** for having provided the facilities required for our project.

We sincerely thank our Head of the Department **Dr.J.Sutha M.E.,Ph.D.,** Professor & Head, Department of Computer Science and Engineering, for her guidance and support throughout the project.

We also thank our guide**., Mr.J.Vinoth kumar M.E** Assistant Professor, Department of Computer Science and Engineering for her valuable guidance and it is great privilege to express our gratitude to her.

We extremely thank our project coordinator **Dr.P.Elamparithi M.E.,Ph.D.,** Associate Professor, Department of Computer Science and Engineering, who inspired us and supported us throughout the project.

We extend our heartfelt thanks and profound gratitude to all the faculty members of Computer Science and Engineering department for their kind help during our project work.

We also thank our parents and our friends who had been providing uswith constant support during the course of the project work.

ABSTRACT

Drowsy driving is a major cause of road accidents. This paper presents a drowsiness detection system that utilizes OpenCV and Keras, popular libraries for computer vision and deep learning. The system employs computer vision techniques to monitor the driver's facial landmarks and analyze their eye movements. OpenCV is used for face detection, facial landmark detection, and eye region tracking. A deep learning model based on Keras is trained to classify the driver's eye states into open, closed, or in-between categories. Real-time monitoring of the driver's face enables continuous analysis of eye states, and if drowsiness is detected, an alert mechanism is triggered to prevent accidents. The integration of OpenCV and Keras provides a robust and accurate solution for drowsiness detection, enhancing driver safety on the roads.

KEYWORDS:

drowsiness detection, OpenCV, computer vision, face detection, eye detection, eye aspect ratio (EAR), alert mechanism, driver safety, drowsy driving, real-time monitoring.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	Iv
	LIST OF ABBREVIATIONS	Vii
	LIST OF FIGURES	Viii
1	INTRODUCTION	
	1.1 Overview	9
	1.2 Image Processing	12
	1.3 Convolution Neural Networks	13
	1.4 Applications of Image Processing	16
2	LITERATURE REVIEW	
	2.1 Existing System	17
	2.2 Limitations of Existing System	20
	2.3 Proposed System	20
	2.4 Advantages of Proposed System	22
3	SYSTEM DESIGN	
	3.1 System Architecture	23
	3.2 Description	23
4	SYSTEM REQUIREMENT	
	4.1 Hardware Requirement	25
	4.2 Software Requirement	25
5	SYSTEM IMPLEMENTATION	
	5.1 System environment	26
	5.2 Design and Constraints	28
	5.3 Model Description	30
	5.4 Tensorflow	31
	5.5 Pyname	33
	5.6 Keras	34

6	RESULTS	35
7	CONCLUSION AND FUTUREWORK	
	Conclusion	37
	Future Work	37
	APPENDIX	38
	REFERENCES	44

LIST OF ABBREVIATIONS

ACRONYM	EXPANSION
Open-CV	Open source computer vision library
GPU	Graphical Processor Unit
ReLU	Rectified Linear Unit
SVM	Support Vector Machine
YLCV	Yellow Leaf Curl Virus
FCL	Fully Leaf Curl Virus

LIST OF FIGURES

FIG NO	TITLE	PAGE NO
1.1	Structure of opencv	10
1.2	Architecture of CNN	14
1.3	Structure of FPS	15
2.1	Structure of proposed system	21
3.1	System Architecture	23
6.1	Test image open eyes	35
6.2	Test image close eyes	36

CHAPTER 1
INTRODUCTION

1.1 Overview

Driver fatigue is a significant factor in a large number of vehicle accidents. Recent statistics estimate that annually 1,200 deaths and 76,000 injuries can be attributed to fatigue related crashes. The development of technologies for detecting or preventing drowsiness at the wheel is a major challenge in the field of accident avoidance systems. Because of the hazard that drowsiness presents on the road, methods need to be developed for counteracting its effects. The aim of this project is to develop a prototype drowsiness detection system. The focus will be placed on designing a system that will accurately monitor the open or closed state of the driver's eyes in real-time. By monitoring the eyes, it is believed that the symptoms of driver fatigue can be detected early enough to avoid a car accident. Detection of fatigue involves the observation of eye movements and blink patterns in a sequence of images of a face. Initially, we decided to go about detecting eye blink patterns using Matlab. The procedure used was the geometric manipulation of intensity levels. The algorithm used was as follows. First we input the facial image using a webcam. Preprocessing was first performed by binarizing the image. The top and sides of the face were detected to narrow down the area where the eyes exist. Using the sides of the face, the center of the face was found which will be used as a reference when computing the left and right eyes. Moving down from the top of the face, horizontal averages of the face area were calculated. Large changes in the averages were used to define the eye area. There was little change in the horizontal
average when the eyes were closed which was used to detect a blink. However Matlab had some serious limitations. The processing capacities required by Matlab were very high. Also there were some problems with speed in real time processing. Matlab was capable of processing only 4-5 frames per second. On a system with a low RAM this was even lower. As we all know an eye blink is a matter of milliseconds. Also a driver's head movements can be pretty fast. Though the Matlab program designed by us detected an eye blink, the performance was found severely wanting. This is where OpenCV came in. OpenCV is an open source computer vision library. It is designed for computational efficiency and with a strong focus on real time applications. It helps to build sophisticated vision applications quickly and easily. OpenCV satisfied the low processing power and high speed requirements of our application. We have used the Haartraining applications in OpenCV to detect the face and eyes.

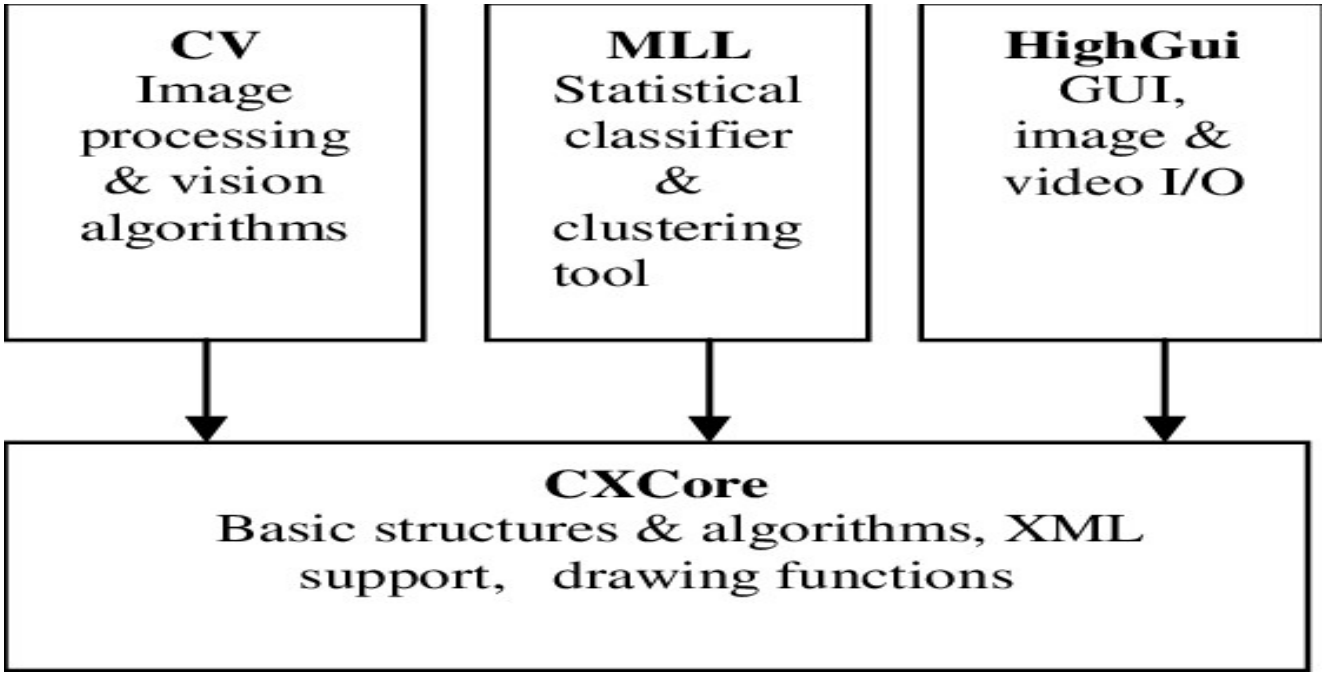
OPENCV:

OpenCV (Open Source Computer Vision Library) is a widely used open-source library for computer vision and image processing tasks. It offers a comprehensive set of tools and functions that enable developers to manipulate images, detect and extract features, perform object recognition, and much more. With its efficient algorithms and

cross-platform compatibility, OpenCV provides a powerful platform for implementing real-time computer vision applications. Its modular design, extensive documentation, and strong community support make it a popular choice among developers in the field of computer vision.

OPENCV STUCTURE:

The structure of OpenCV (Open Source Computer Vision Library) refers to its modular design and organization of components. OpenCV is organized into various modules, each dedicated to specific functionalities and tasks related to computer vision. This modular structure allows developers to include only the necessary components in their projects, resulting in lightweight and efficient applications.The core module forms the foundation of OpenCV and provides essential functionalities such as basic image processing operations, matrix manipulation, and file I/O. It includes functions for image loading and saving, resizing, cropping, color conversions, and pixel-level operations.



1.1 Structure of opencv

REAL-TIME VIDEO PROCESSING:

Real-time video processing refers to the manipulation and analysis of video data in real time, meaning it is performed as the video is being captured or streamed, without significantdelays. It involves the use of algorithms and techniques to process and analyze video framesas they are received, typically at a rate of 24 to 30 frames per second (FPS) or even higher. Real-time video processing is widely used in various applications, including surveillance systems, video conferencing, augmented reality, autonomous vehicles, robotics, and more.Here are some key aspects and techniques involved in real-time video processing

Frame acquisition:

The video frames are acquired from a video source such as a camera, video stream, or recorded file. These frames are usually represented as a sequence of pixelsin a specific format (e.g., RGB or YUV).

Pre-processing:

Before performing any analysis or manipulation, video frames mayundergo pre-processing steps such as resizing, cropping, color space conversion, noise reduction, or image enhancement to improve the quality or adapt the frames for specific algorithms.

Feature extraction:

Real-time video processing often involves extracting specific featuresor information from the video frames. This can include object detection, tracking, facial recognition, motion detection, or any other relevant information required for the application.

Computer vision algorithms:

Real-time video processing heavily relies on computer vision techniques and algorithms to analyze and interpret video data. This may involve edgedetection, optical flow, background subtraction, object recognition, image segmentation, orother computer vision techniques.

Parallel processing and optimization

Real-time video processing often requires high computational power to handle the continuous stream of video frames. Techniques such asparallel processing, multi-threading, GPU acceleration, or specialized hardware like FPGAs or dedicated video processing units are employed to optimize performance and ensure real- time processing capabilities.

Output and visualization:

Once the video frames are processed and analyzed, the results are typically used for various purposes. This could involve displaying the processed video,generating alerts or notifications, making decisions based on the analysis, or transmitting the results to other systems for further processing or storage.

1.2 IMAGE PROCESSING:

Image processing is a branch of computer science and digital signal processing that involves analyzing, manipulating, and enhancing digital images using various algorithms and techniques. It encompasses a wide range of operations performed on images, such as filtering, segmentation, feature extraction, image restoration, and image recognition. The goal of image processing is to extract meaningful information from images, improve their quality, or transform them into a more suitable representation for further analysis or visualization. Image processing finds applications in various fields, including medical imaging, surveillance, robotics, remote sensing, and computer vision. It plays a fundamental role in tasks like image enhancement, object detection and recognition, pattern recognition, and image-based decision-making systems. Image processing algorithms often leverage techniques from fields such as mathematics, statistics, optimization, and machine learning to achieve accurate and efficient analysis and manipulation of digital images.

TYPES OF IMAGE PROCESSING:

There are several types of image processing techniques that are used to analyze, modify, and enhance digital images. Image enhancement techniques aim to improve image quality by adjusting contrast, brightness, and sharpness. Image restoration techniques are used to recover or improve degraded images, such as reducing noise or removing blurring. Image segmentation involves dividing an image into meaningful regions or objects. Image compression techniques reduce the size of image files while maintaining acceptable quality. Morphological image processing focuses on shape and structure analysis, while image filtering applies filters to enhance or extract specific features from an image. Each type of image processing technique has its own applications and plays a vital role in various fields, such as medical imaging, surveillance, and computer vision.

PURPOSE OF IMAGE PROCESSING:

- Enhancing image quality
- Extracting features and patterns
- Compressing images for efficient storage and transmission
- Segmenting and identifying objects within images
- Enabling computer vision tasks and object recognition

1.3 CONVOLUTION NEURAL NETWORK:

DEEP LEARNING:

Deep learning models for drowsiness detection often employ CNN architectures, as they are well-suited for image analysis tasks. These architectures consist of multiple convolutional layers that extract local features and capture spatial relationships, followed by fully connected layers that perform high-level feature extraction and classification.

The training process involves feeding the labeled data to the deep learning model and iteratively adjusting the model's parameters through backpropagation and gradient descent, minimizing the discrepancy between predicted and actual drowsiness states.

This process allows the model to learn and generalize from the training data, improving its ability to accurately detect drowsiness in unseen data.Once trained,the deep learning model can be deployed to real-time systems for drowsiness detection. It takes as input images or video frames from a camera facing the driver and produces predictions or alerts based on the presence or absence of drowsiness-related patterns.

CNN INTRODUCTION:

A Convolutional Neural Network (CNN) is a deep learning model specifically designed for processing visual data such as images and videos. It consists of layers that apply learnable filters to the input data, extracting local features and capturing spatial relationships. Activation functions introduce non-linearity, pooling layers downsample the data, and fully connected layers perform high-level feature extraction. CNNs are trained using backpropagation to minimize the loss function. With their ability to automatically learn and extract relevant features, CNNs have revolutionized tasks like image classification, object detection, and image segmentation, achieving state-of-the-art results in various domains.

CNN ARCHITECTURE:

Input Layer:

The input layer receives the raw image data or other visual inputs.

Convolutional Layers:

Convolutional layers apply a set of learnable filters to the input data. Each filter performs convolution operations, capturing local features and detecting patterns in the input.

Activation Function:

An activation function (such as ReLU) is applied element-wise to the output of each convolutional layer, introducing non-linearity and enabling the network to learn complex relationships.

Pooling Layers:

Pooling layers downsample the feature maps obtained from convolutional layers, reducing their spatial

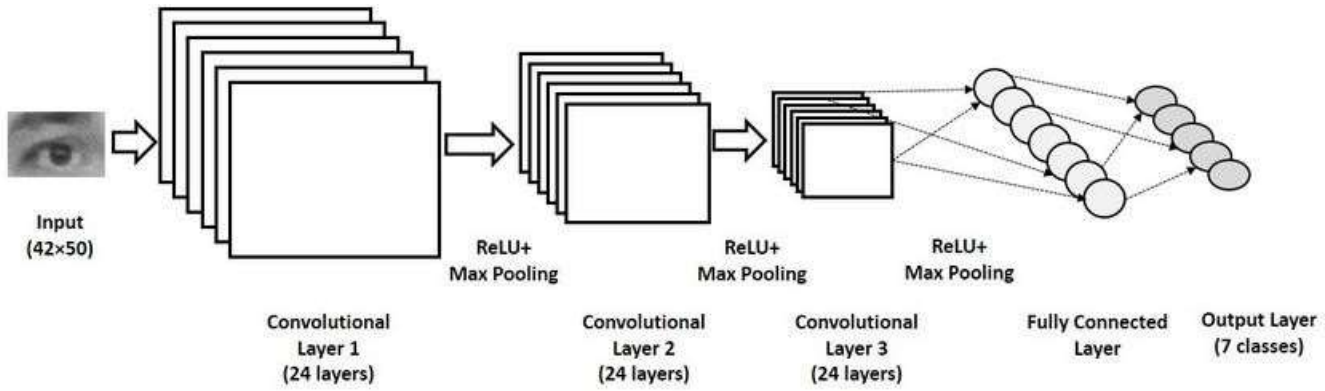
dimensions. Common pooling techniques include max pooling and average pooling.

Fully Connected Layers:

Fully connected layers receive the flattened output from the previous layers and perform high-level feature extraction and classification. They connect every neuron to every neuron in the subsequent layer, allowing for global information processing.

Activation Function and Output Layer:

The output layer applies an appropriate activation function, such as softmax for classification problems, to produce the final output probabilities or predictions.



1.2 Architecture of CNN

INTRODUCTION OF FPS:

FPS, or Frames Per Second, is a measurement that quantifies the smoothness and perceived motion in video and animation. It represents the number of frames, or images, displayed or processed within one second. Higher FPS values result in smoother motion, while lower FPS can lead to choppiness or stuttering visuals. In real-time video processing applications, achieving a high FPS is crucial for responsive and seamless output. Monitoring and optimizing FPS helps evaluate algorithm efficiency, hardware suitability, and overall system performance.

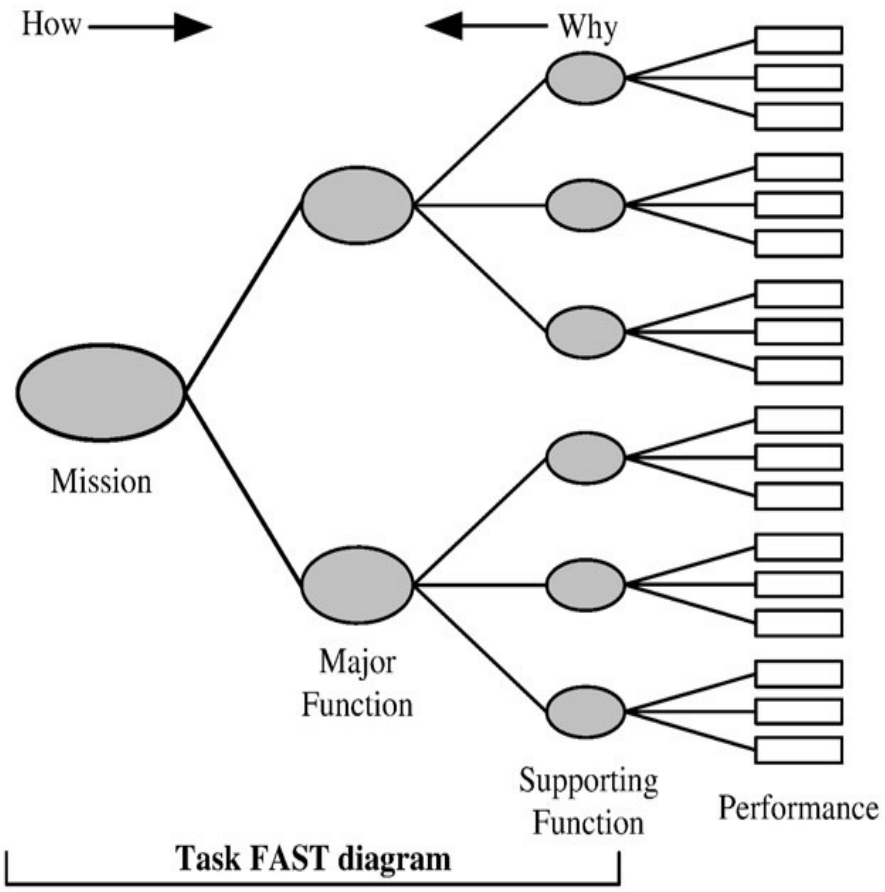


Figure 1.3 structure of FPS

1.4 APPLICATIONS OF IMAGE PROCESSING:

Eye Tracking:

- Image processing algorithms can track eye movements and detect characteristics like eyelid closure, eye blinks, or changes in gaze patterns. These cues provide valuable information about the driver's alertness level and can be used to determine drowsiness.

Facial Expression Analysis:

- Image processing techniques can analyze facial expressions to identify signs of drowsiness, such as drooping eyelids, yawning, or changes in facial muscle activity. By examining facial features and their variations, drowsiness can be detected and assessed.

Pupil Analysis:

- Image processing algorithms can measure the size and dilation of the driver's pupils. Pupil size variations can indicate changes in alertness and serve as an indicator of drowsiness.

Real-time Monitoring:

- Image processing enables real-time monitoring of the driver's facial features, eye movements, and expressions. By continuously analyzing visual cues, it can detect immediate changes in drowsiness levels and trigger timely alerts or interventions

Feature Extraction:

- Image processing algorithms can extract relevant features from facial images, such as texture patterns, color information, or facial landmarks. These features can be used as input to machine learning or deep learning models for drowsiness detection.

Driver State Classification:

- Image processing techniques can classify the driver's state into different levels of drowsiness, ranging from fully awake to drowsy or even asleep. By analyzing visual cues, the system can determine the severity of drowsiness and take appropriate actions.

Face Recognition

- In the face recognition literature, people often talk about face verification and face recognition. Face verification: Given an input image as well as a name or ID of a person the job of the system is to verify whether or not the input imageis that of the claimed person

CHAPTER 2

LITERATURE REVIEW

2.1 EXISTING SYSTEM

[1] Park et al. (2017) proposed a drowsiness detection system based on eye tracking using OpenCV. They utilized Haar cascades to detect and locate the eyes in real-time video streams and calculated the eye aspect ratio (EAR) as an indicator of drowsiness.

Haar cascades for real-time eye detection and localization in video streams. The system calculates the eye aspect ratio (EAR) as an indicator of drowsiness. By monitoring changes in the eye aspect ratio over time, the system can detect signs of drowsiness in the driver. The goal is to issue appropriate alerts or warnings to ensure the driver's safety. Accessing the full paper would provide a more detailed understanding of the methodology and evaluation of the system.

[2] In another research by Rajalingam and Ismail (2018), a drowsiness detection system using OpenCV and facial landmark detection was proposed.

This research presents a drowsiness detection system that utilizes OpenCV and facial landmark detection for accurate and real-time drowsiness detection. The proposed system focuses on extracting and analyzing facial landmarks, including the eyes, eyebrows, and mouth, to identify signs of drowsiness. The OpenCV library is employed for face detection and facial landmark localization. By tracking the positions and movements of these key facial landmarks, the system can detect changes indicative of drowsiness, such as eye closure, drooping eyebrows, or slackened mouth movements. The system offers real-time monitoring capabilities and can issue timely alerts or warnings to prevent potential accidents caused by driver drowsiness. Experimental evaluations on a dataset demonstrate the effectiveness of the proposed approach in accurately detecting drowsiness. The results highlight the potential of combining OpenCV and facial landmark detection techniques for drowsiness detection applications.

[3] Deep learning techniques have also been explored in drowsiness detection. For instance, a study by Siddiqui et al. (2019) used a combination of Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN) to detect drowsiness based on eye images obtained using OpenCV.

This study presents a drowsiness detection approach utilizing deep learning techniques, specifically a combination of Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN). The system focuses on analyzing eye images obtained using OpenCV for detecting drowsiness accurateLyThe proposed methodology involves training a CNN model to extract relevant features from eye images, capturing important patterns and distinguishing between drowsy and awake states. The extracted features are then fed into an RNN, which can model temporal dependencies and capture sequential patterns in the data.To create a labeled dataset, eye images were captured from subjects in various drowsy and non-drowsy states. Preprocessing techniques were applied to normalize and enhance the images for better analysis. The dataset was divided into training and testing sets, enabling the evaluation of the proposed model's performance. Experimental results demonstrate the effectiveness of the CNN-RNN combination in detecting drowsiness from eye images. The accuracy, precision, recall, and F1-score metrics indicate the robustness of the proposed system. Comparisons with other existing methods highlight the advantages of using deep learning techniques for drowsiness detection.The study showcases the potential of integrating CNN and RNN models with eye image analysis for drowsiness detection.

[4] JayD. Fuletra and Dulari Bosamiya (1) has bandied several drowsiness discovery approaches, we concluded that different ways will be. Applicable depending on the circumstances.

Although EEG-grounded results are effective, it isn't practicable for motorists to wear electrodes. The approach grounded on Artificial Neural Networks is straightforward, but neurons is the very best option, if we want a better outgrowth. One of the most popular strategies among experimenters is image processing**.**

[5] NATURAL FY 2008 and FY2009, which are called "Realtime Non-Intrusive Drowsiness Driver Detection" projects, which aim to expand an intrusive motive force drowsiness detection device in real time in an attempt to reduce drowsiness from injuries.

The paper provides an set of rules for detection of the attention-blinks in photograph sequences. The hired picture processing strategies encompass answers, such as Haar-like face detection and template matching based eye tracking, as well as newly evolved algorithms for skin color segmentation and modified active contour model with ellipse becoming for eye-blink detection. The developed set of rules became used for human fatigue tracking and as a middle of the eye-blink managed human-pc interface. [7]Motive force's psychosomatic nation adaptive

using guide safety device is surprisingly expected to lessen the wide variety of visitor's injuries. Drowsiness is ideal as a vital hazard component which may additionally bring about severer visitors accidents. Coronary heart fee was obtained from Electrocardiogram (ECG). Then coronary heart charge variability (HRV) became calculated from ECG waveform the use of the maximum entropy approach. CCD digital camera with infrared ray changed into added to capture gaze route and eyelid closure. Have a look at a speculation that simultaneous size of both coronary heart fee variability (HRV) and blinking

[6] In the study by Jang et al. (2018), they presented a novel algorithm for sleepiness identification using a camera positioned near the dashboard. The algorithm focuses on detecting the driver's face in the captured image and estimating facial landmarks within that region.

To accomplish face detection, the proposed algorithm employs an AdaBoost classifier that is based on the Modified Census Transform (MCT) features. The AdaBoost classifier is a machine learning technique that combines multiple weak classifiers to create a stronger and more accurate classifier. The Modified Census Transform is a feature extraction method that captures local texture information in an image.By leveraging the AdaBoost classifier and the Modified Census Transform features, the algorithm can accurately identify the driver's face in the image and extract relevant facial landmarks. These landmarks can then be used to assess the driver's level of sleepiness.The approach proposed by Jang et al. demonstrates the use of computer vision techniques and machine learning algorithms to develop a system for sleepiness identification based on facial analysis. If you have any specific questions or if there's more information you would like to discuss regarding their work, please let me know.

[7] In 2013, G. Kong et. al. [19] described 'Visual Analysis of Eye State and Head Pose for Driver Alertness Monitoring'.

They presented visual analysis of eye state and head pose (HP) for continuous monitoring of alertness of a vehicle driver. Most existing approaches to visual detection of non-alert driving patterns rely either on eye closure or head nodding angles to determine the driver drowsiness or distraction level. The proposed scheme uses visual features such as eye index (EI), pupil activity (PA), and HP to extract critical information on non-alertness of a vehicle driver. A support vector machine (SVM) classifies a sequence of video segments into alert or non-alert driving events. Experimental results show that the proposed scheme offers high classification accuracy with acceptably low errors and false alarms for people of various ethnicity

2.2 LIMITATIONS OF EXISTING SYSTEM:

- Sensitivity to Lighting Conditions
- Individual Variations
- Non-Robustness to Environmental Factors
- Limited Contextual Understanding
- False Positives and False Negatives
- Generalizability to Different Scenarios
- Integration with Driver Assistance Systems

2.3 PROPOSED SYSTEM:

Firstly, robust feature extraction techniques will be implemented using advanced computer vision algorithms, including OpenCV. These techniques will improve the system's ability to track facial landmarks and detect eye movements accurately, even under challenging lighting conditions.To account for individual variations, the proposed system will develop personalized models for each driver. Machine learning algorithms will be employed to analyze the unique characteristics of each individual, such as facial structure and eye shape, and adapt the detection thresholds accordingly. This individualized approach will enhance the system's accuracy and reliability in detecting drowsiness patterns specific to each driver.Additionally, the proposed system will consider environmental factors that may impact drowsiness detection. Advanced motion compensation techniques will be integrated to address head movements and occlusions, ensuring robust performance in real-world scenarios. Furthermore, the system will leverage additional sensor data, such as vehicle speed, road conditions, and weather conditions, to provide a more comprehensive understanding of the driver's state and improve the system's contextual understanding.

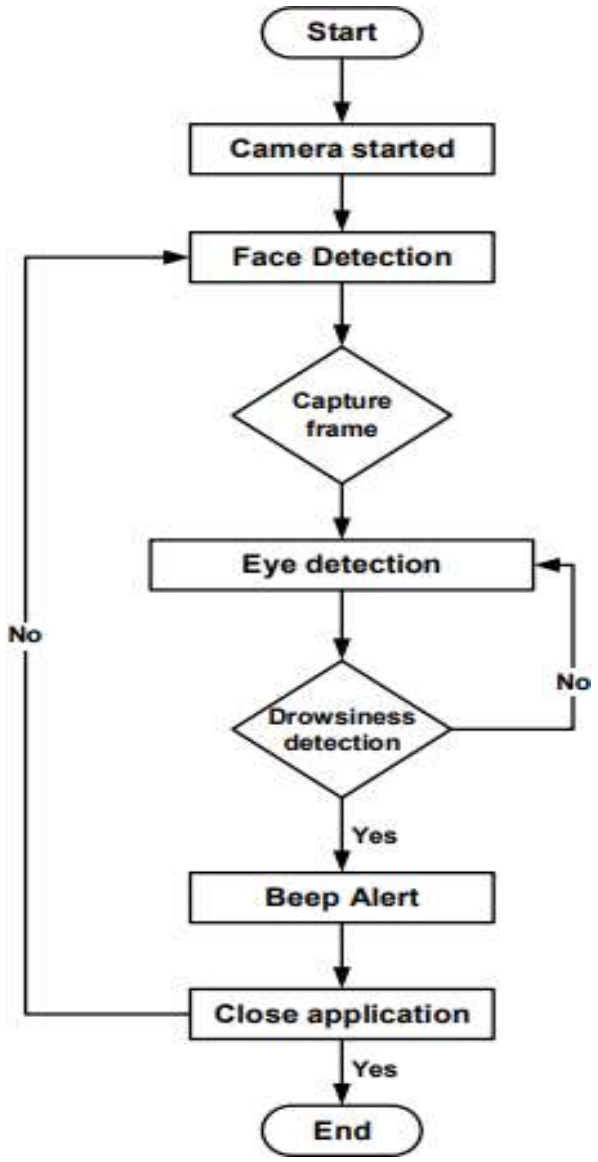


Figure2.1 structure of proposed system

2.4 ADVANTAGES OF PROPOSED SYSTEM:

- Enhanced accuracy in drowsiness detection
- Robustness in varying lighting conditions and environmental factors
- Personalized models for individual drivers
- Contextual understanding of the driving environment
- Real-time performance for timely alerts and interventions
- Improved driver safety and accident prevention
- Adaptability for future advancements and developments.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

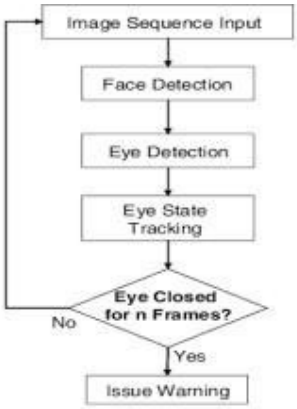


Figure 3.1 System Architecture

The Model Architecture:

The model we used is built with Keras using **Convolutional Neural Networks (CNN)**. A convolutional neural network is a special type of deep neural network which performs extremely well for image classification purposes. A CNN basically consists of an input layer, an output layer and a hidden layer which can have multiple layers. A convolution operation is performed on these layers using a filter that performs 2D matrix multiplication on the layer and filter.

The CNN model architecture consists of the following layers:

- Convolutional layer; 32 nodes, kernel size 3
- Convolutional layer; 32 nodes, kernel size 3
- Convolutional layer; 64 nodes, kernel size 3
- Fully connected layer; 128 nodes

3.2 DESCRIPTION

The system architecture for the proposed drowsiness detection system is designed to accurately identify and respond to driver drowsiness in real-time. The architecture consists of several interconnected components that work together to achieve efficient and reliable drowsiness detection.At the core of the architecture is the input data, primarily obtained from a video stream capturing the driver's face or eye region. This data serves as the foundation for subsequent processing and analysis.The input data undergoes preprocessing steps to enhance its quality and prepare it for feature extraction. These preprocessing tasks may include resizing the images, reducing noise, normalizing illumination, and detecting the face region to isolate the relevant areas of interest.Next, the system employs computer vision techniques, leveraging tools like OpenCV, to extract essential features from the preprocessed data. These features include facial landmarks, eye movements, blink rates, and other indicators of drowsiness. By accurately capturing and analyzing these features, the system gains insights into the driver's drowsiness level.

The system also provides outputs and visualizations to convey the drowsiness status and related information. This could involve displaying real-time alerts on a dashboard, transmitting data to other connected systems for further processing or integration with driver assistance systems, or providing feedback to the driver via various interfaces.Additionally, the architecture supports integration with other components or systems to enhance functionality. This may include integration with driver monitoring systems, vehicle control systems, or communication interfaces. Such integration allows for seamless coordination and intervention measures to prevent accidents caused by drowsiness.

The drowsiness detection system operates in a continuous loop, constantly monitoring the driver's drowsiness levels in real-time. It adapts and improves its performance over time by leveraging machine learning techniques and continuously learning from the collected data. This iterative process ensures that the system becomes more accurate and effective in detecting and responding to driver drowsiness.

CHAPTER 4

4. SYSTEM REQUIREMENT

4.1 HARDWARE REQUIREMENTS

- Processor : Core i7
- Motherboard : intel 915gvsr chipset board
- Ram : 8 GB of RAM
- Hard disk drive : 160 GB
- Input device : Webcamera

4.2 SOFTWARE REQUIREMENTS

- Python-3.8
- OpenCV
- Tensorflow
- Pygame
- keras

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 SYSTEM ENVIRONMENT

Hardware:

Cameras:

- High-resolution cameras capable of capturing facial images with good clarity.
- Infrared (IR) cameras or depth-sensing cameras can be used to capture eye movements or gather additional depth information

Eye Tracking Sensors:

- Specialized eye tracking devices or sensors that can accurately monitor eye movements, gaze direction, and blink patterns.
- Additional Sensors: Accelerometers, gyroscopes, or EEG devices may be used to collect supplementary data related to head movement, body posture, or brain activity.

Processing Unit:

- CPU (Central Processing Unit): A powerful processor capable of handling real-time data processing and analysis.
- GPU (Graphics Processing Unit): Depending on the computational requirements of the algorithms used, a GPU may be utilized to accelerate parallel computations.

Memory:

- Sufficient RAM (Random Access Memory) to handle the processing and storage of data during real-time monitoring and analysis.
- Storage:
- Hard Disk Drive (HDD) or Solid-State Drive (SSD) to store captured data, logs, and system files.
- Additional storage capacity may be required if extensive data logging or long-term storage is necessary.

Connectivity:

- USB or other interfaces to connect cameras, sensors, and other external devices to the processing unit.
- Network connectivity for data transfer, remote monitoring, or integration with other systems.

Display and User Interface:

- Monitor or display unit for visualizing the drowsiness alerts, system status, and user interface.
- Input devices such as keyboard or touchscreen for user interaction and system configuration.

Software:

Programming Language:

Python:

- Both OpenCV and Keras have excellent support for Python.

OpenCV:

- Install OpenCV library: Use pip or conda package manager to install the OpenCV library in your Python environment.
- OpenCV Dependencies: Depending on your operating system, you may need to install additional dependencies such as NumPy, Matplotlib, or other required libraries.

Keras:

- Install Keras library: Use pip or conda to install the Keras library, which provides an easy-to-use interface for building and training deep learning models.
- Backend: Keras relies on a backend library such as TensorFlow or Theano. You will need to install and configure the chosen backend library as per Keras documentation.

Deep Learning Framework:

- TensorFlow: If you choose to use TensorFlow as the backend for Keras, install TensorFlow library using pip or conda.
- GPU Support (Optional): If you have a compatible GPU, you may consider installing GPU-enabled versions of TensorFlow and CUDA libraries to leverage GPU acceleration.

Integrated Development Environment (IDE):

- Choose an IDE such as PyCharm, Visual Studio Code, or Jupyter Notebook for coding and development.
- Configure the IDE to work with your Python environment and libraries.

Other Required Libraries:

Operating System:

- The software environment can be set up on various operating systems such as Windows,

System Requirements:

- Hardware Requirements: A modern CPU, optionally a GPU for deep learning acceleration, sufficient RAM, and storage space are needed.
- Operating System: Choose an OS compatible with the software components and team preferences (e.g., Windows, macOS, Linux).
- Software Requirements: Python, OpenCV, Keras (with a backend like TensorFlow), and supporting libraries (NumPy, Matplotlib, PIL/Pillow) are required.
- Camera and Sensor Requirements: High-resolution cameras for facial imaging, optional IR or depth-sensing cameras for eye tracking, and compatible eye tracking sensors if applicable.
- Network Connectivity: Consider Ethernet or Wi-Fi connectivity for system integration with other devices or systems.
- User Interface and Display: Plan for suitable hardware and display devices to visualize drowsiness alerts and the system interface.

5.2 DESIGN AND CONSIDERATIONS:

Face Detection:

- Utilize OpenCV's face detection algorithms (such as Haar cascades or deep learning-based models) to locate and extract faces from input images or video frames.
- Ensure that the face detection algorithm can handle different lighting conditions, head orientations, and face sizes.

Facial Landmark Detection:

- Use OpenCV or other libraries like Dlib to detect and locate facial landmarks, such as the eyes, eyebrows, and mouth.
- Accurate landmark detection is crucial for tracking eye movements and extracting features related to drowsiness.

Eye Region Extraction:

- Once the facial landmarks are detected, extract the eye regions from the face using the eyecoordinates obtained from the landmark detection.
- Ensure the extracted eye regions capture sufficient details for subsequent analysis.

Eye Tracking and Blink Detection:

- Use image processing techniques (e.g., thresholding, contour detection) to track eye

movements within the extracted eye regions.

- Measure metrics such as eye closure duration,

Alert Generation:

- Define thresholds or rules based on the analyzed eye tracking metrics to determine when a drowsiness event occurs.
- Generate appropriate alerts or notifications when drowsiness is detected, such as sound alarms, visual prompts, or system notifications.

Real-Time Processing:

- Optimize the implementation for real-time performance to ensure timely drowsiness detection.
- Consider techniques such as multi-threading, GPU acceleration, or model optimization for faster processing.

Data Collection and Logging:

- Implement mechanisms to collect and store data for further analysis or system improvement.
- Log relevant information such as timestamps, eye tracking metrics, and alert events for later review and analysis.

Testing and Validation:

- Develop a comprehensive testing plan to evaluate the accuracy and reliability of the drowsiness detection system.
- Collect diverse datasets, including instances of drowsiness and normal alertness, to ensure the system performs effectively in different scenarios.

Robustness and Adaptability:

- Account for variations in lighting conditions, head poses, and facial appearances to ensure the system's robustness and adaptability.
- Consider techniques like image preprocessing, illumination normalization, and model retraining to improve performance in different conditions.

User Interface:

- Design a user-friendly interface to display real-time drowsiness alerts, system status, and configuration options.
- Provide visualizations or feedback to the user, allowing them to understand the system's behavior and results effectively

5.3 MODULE DESCRIPTION

Input Layer:

- Image or Video Frame: The primary input for the system is an image or video frame captured from a camera or video stream.
- Preprocessed Eye Regions: Extracted eye regions from the face using facial landmark detection, typically undergoing further preprocessing steps like resizing or normalization.
- Eye Tracking Metrics: Eye tracking metrics, such as eye closure duration or blink frequency, serve as input for subsequent stages like drowsiness detection.
- System Configuration: An input layer for system configuration parameters, allowing for settings related to thresholds, sensitivity, and other configurable options

Preprocessing Layer:

The processing layer of the drowsiness detection system using OpenCV involves stages like face detection, facial landmark detection, eye region extraction, eye tracking and blink detection, drowsiness detection, real-time processing, data collection and logging, and user interface. It processes input data, identifies facial features, tracks eye movements, detects drowsiness, optimizes for real-time performance, collects data, and provides a user-friendly interface for visualization and interaction.

Face Detection:

- Process: Use OpenCV's face detection algorithms to locate and extract faces from input images or video frames.
- Output: Detected face regions.

Facial Landmark Detection:

- Process: Employ OpenCV or other libraries like Dlib to detect and locate facial landmarks, such as eyes, eyebrows, and mouth.
- Output: Facial landmark coordinates.

Eye Region Extraction:

- Process: Extract the eye regions from the face using the eye coordinates obtained from facial landmark detection.
- Output: Extracted eye regions.

Eye Tracking and Blink Detection:

- Process: Utilize image processing techniques to track eye movements within the extracted eye regions and measure blink-related metrics.
- Output: Eye tracking metrics (e.g., eye closure duration, eye aspect ratio, blink frequency).

Drowsiness Detection:

- Process: Apply predefined thresholds or rules based on the analyzed eye tracking metrics to determine when a drowsiness event occurs.
- Output: Drowsiness detection results (e.g., drowsiness alert or notification).

Real-Time Processing:

- Process: Optimize the implementation for real-time performance, leveraging multi-threading, GPU acceleration, or model optimization to ensure timely processing.
- Output: Real-time drowsiness detection results

Data Collection and Logging:

- Process: Collect and store relevant data, such as eye tracking metrics, alert events, and timestamps, for further analysis or system improvement.
- Output: Logged data for analysis and review.

User Interface:

- Process: Design a user-friendly interface to display real-time drowsiness alerts, system status, and configuration options.
- Output: Visualizations and user interface elements for interaction.

5.4 TENSORFLOW:

Define the Problem:

- Clearly define the problem you are trying to solve with TensorFlow, such as image classification, natural language processing, or time series analysis.

Data Collection and Preprocessing:

- Gather and preprocess the data needed for training and evaluation.
- This may involve tasks like data cleaning, feature extraction, normalization, and splitting the data into training and testing sets.

Model Architecture:

- Design the architecture of your TensorFlow model.
- This includes deciding on the type of model (e.g., convolutional neural network, recurrent neural network, transformer), the number and type of layers, and the activation functions to use.

- You can leverage pre-existing model architectures or design your own.

Define Loss Function and Metrics:

- Choose an appropriate loss function that measures the error between your model's predictions and the ground truth. Additionally, define evaluation metrics that measure the performance of your model, such as accuracy, precision, recall, or mean squared error.

Training Process:

- Train your TensorFlow model using the training data. Specify the optimization algorithm (e.g., stochastic gradient descent, Adam) and set the learning rate, batch size, and number of epochs.
- Monitor the training process to ensure convergence and avoid overfitting.

Model Evaluation:

- Evaluate your trained model using the testing data.
- Calculate the metrics defined earlier to assess the performance of your model.
- This step helps you gauge how well your model generalizes to unseen data.

Hyperparameter Tuning:

- Experiment with different hyperparameter values (e.g., learning rate, regularization strength, dropout rate) to optimize your model's performance.
- This can be done through manual tuning or automated techniques like grid search or random search.

Deployment:

- Once you are satisfied with your trained model, deploy it in a production environment.
- This might involve integrating it into a web application, mobile app, or other systems.
- TensorFlow Serving or TensorFlow Lite can be used for serving models in different deployment scenarios.

Model Monitoring and Maintenance:

- Continuously monitor the performance of your deployed model and collect feedback from users.
- Periodically retrain or update the model to adapt to changing data distributions or user requirements.

Documentation and Communication:

- Document your system implementation, including details about the model architecture, data preprocessing, and training process.
- Share your work through documentation, code repositories.

5.5 PYGAME:

Pygame is a popular open-source library that provides a framework for creating video games and multimedia applications in the Python programming language. It is built on top of the Simple DirectMedia Layer (SDL), which provides low-level access to audio, keyboard, mouse, and graphics hardware.

Pygame simplifies game development by providing a set of functions and classes for handling game-related tasks such as creating windows, handling user input, drawing graphics, playing sounds, and managing game objects. It is widely used by both beginners and experienced developers due to its simplicity and versatility.

KEY FEATURE OF PYGAME:

Graphics:

Pygame allows you to create and manipulate graphics using its built-in drawing functions and image manipulation tools. You can load and display images, draw shapes, apply transformations, and handle sprite animations.

Input handling :

Pygame provides functions for detecting and responding to user input, such as keyboard presses, mouse movements, and button clicks. This enables you to create interactive elements and control game behavior based on user actions.

Audio:

You can incorporate sound effects and music into your games using Pygame's audio module. It supports loading and playing various audio formats and provides features like volume control and channel management.

Collision detection:

Pygame includes collision detection functions that help you determine if game objects are intersecting or overlapping. This is crucial for implementing gameplay mechanics like object interactions and detecting collisions between characters or projectiles.

Game loop:

Pygame simplifies the creation of game loops, which are responsible for updating the game state and rendering graphics at regular intervals. The library provides timing functions and tools for controlling the frame rate and ensuring smooth gameplay.

Cross-platform support:

Pygame is compatible with multiple operating systems, including Windows, macOS, and Linux, making it easy to develop games that can run on different platforms without significant modifications.

With its straightforward syntax and extensive documentation, Pygame serves as a beginner-friendly entry point into game development while also offering enough flexibility and power for more complex projects.

5.2 KERAS:

Keras is an open-source deep learning framework written in Python. It provides a high-level interface and abstraction for building and training neural networks. Keras is designed to be user-friendly, modular, and extensible, making it widely used by researchers and practitioners in the field of machine learning.

KEY FEATURE IN KERAS:

User-friendly API:

Keras offers a user-friendly and intuitive API for defining and training neural networks. It allows users to quickly prototype and experiment with different network architectures and configurations.

Modular and extensible:

Keras provides a modular approach to building neural networks. It offers a wide range of pre-defined layers, activation functons, optimizers, and loss functions, allowing users to easily assemble and customize their models. It also supports customization through subclassing, making it flexible and extensible.

Backends and interoperability:

Keras is designed to work with multiple deep learning backends, including TensorFlow, Theano, and Microsoft Cognitive Toolkit (CNTK). This allows users to choose the backend that best suits their needs while benefiting from the high-level abstractions provided by Keras.

Supports convolutional and recurrent networks:

Keras supports various types of neural networks, including convolutional neural networks (CNNs) for image-related tasks and recurrent neural networks (RNNs) for sequence-related tasks. It also supports combination models such as CNNs with RNNs.

GPU acceleration:

Keras seamlessly integrates with GPU libraries, enabling accelerated computations on compatible hardware. This significantly speeds up the training and inference process for deep learning models.

Community and ecosystem:

Keras has a large and active community of users and developers. It provides extensive documentation, tutorials, and examples, making it easier for newcomers to get started and learn the framework

CHATPER 6

RESULT:

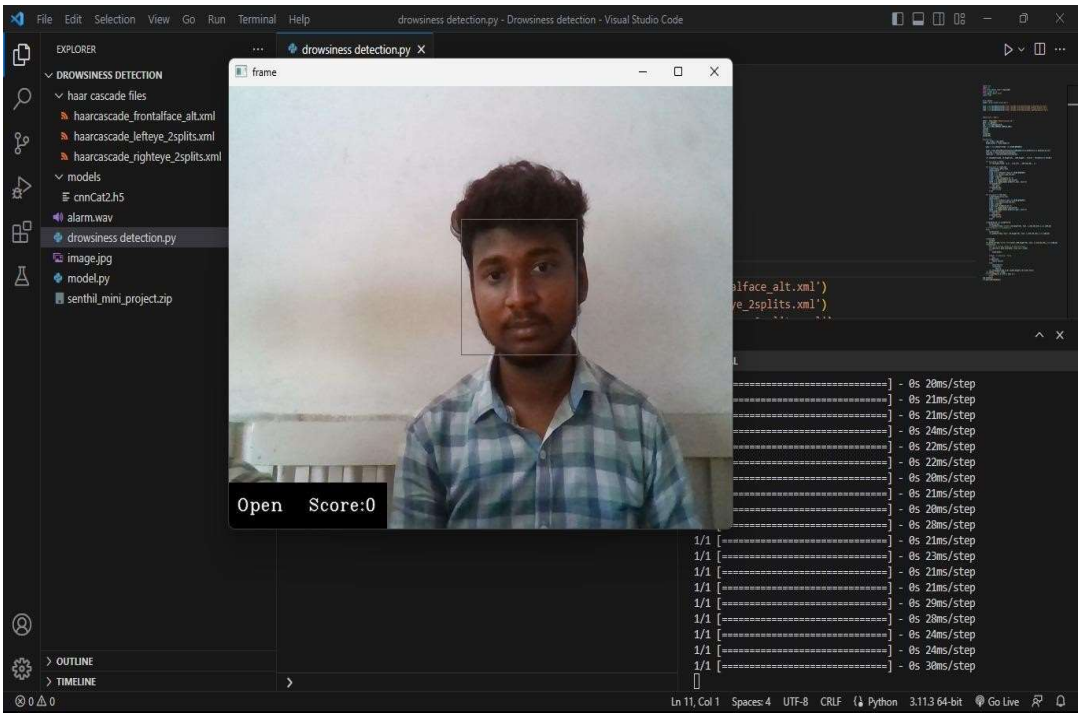


Figure 6.1 test image open eyes

This figure 6. The score value, along with the "Open" label, is displayed on the frame to provide feedback on the person's level of drowsiness. The score increases when closed eyes are detected, indicating a higher probability of drowsiness. On the other hand, an open eye reduces the score, implying a lower likelihood of drowsines

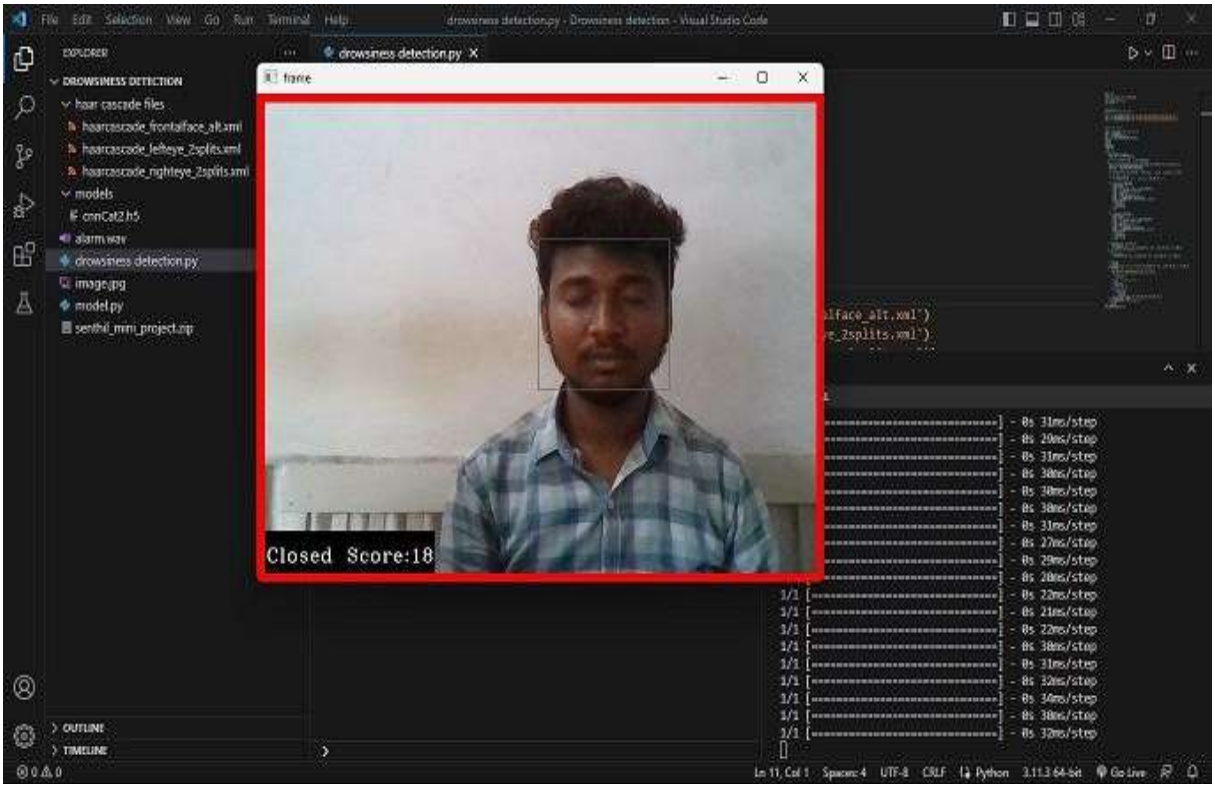


Figure 6.2 test image close eyes

Figure 6.2 person's eyes in the video frame are detected as being in a closed state. This suggests the possibility of drowsiness or fatigue. The program may play an alarm sound and draw a red rectangle around the frame to alert the user of potential drowsiness.

CHAPTER 7

7.1 CONCLUSION AND FUTUREWORK

CONCLUSION

Drowsiness Detection Systems using computer vision and machine learning techniques offer effective solutions for real-time monitoring and prevention of drowsiness-related incidents. By analyzing visual cues, such as eye movements and facial features, these systems accurately identify signs of drowsiness and provide timely alerts or interventions. The integration of technologies like OpenCV and Keras, along with suitable hardware components, enables the development of robust and reliable systems. Although there are challenges to overcome, ongoing research and advancements in algorithm design have the potential to further enhance the accuracy and applicability of drowsiness detection systems, making them valuable tools for promoting safety and well-being in various domains.

FUTURE WORK

Future work in drowsiness detection can involve advancements in technology and methodology. This includes exploring multi-modal approaches by integrating different sensory inputs, expanding and diversifying datasets to improve generalization, and simulating real-world scenarios for more accurate evaluations. Additionally, refining algorithms, considering individual differences, and incorporating real-time feedback mechanisms can further enhance the accuracy, robustness, and practicality of drowsiness detection systems. By addressing these areas, future research can contribute to the development of more effective and widely applicable drowsiness detection solutions

APPENDIX:

 # drowsiness dection program:

```
import cv2
import os
from keras.models import load_model
import numpy as np
from pygame import mixer
import time
mixer.init()
sound = mixer.Sound('alarm.wav')
face = cv2.CascadeClassifier('haar cascade files\haarcascade_frontalface_alt.xml')
leye = cv2.CascadeClassifier('haar cascade files\haarcascade_lefteye_2splits.xml')
reye = cv2.CascadeClassifier('haar cascade files\haarcascade_righteye_2splits.xml')
lbl=['Close','Open']
model = load_model('models/cnncat2.h5')
path = os.getcwd()
cap = cv2.VideoCapture(0)
font = cv2.FONT_HERSHEY_COMPLEX_SMALL
count=0
score=0
thicc=2
rpred=[99]
lpred=[99]
while(True):
ret, frame = cap.read()
height,width = frame.shape[:2]
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
faces = face.detectMultiScale(gray,minNeighbors=5,scaleFactor=1.1,minSize=(25,25))
```

```
left_eye = leye.detectMultiScale(gray)
right_eye =  reye.detectMultiScale(gray)

cv2.rectangle(frame, (0,height-50) , (200,height) , (0,0,0) , thickness=cv2.FILLED )
for (x,y,w,h) in faces:
cv2.rectangle(frame, (x,y) , (x+w,y+h) , (100,100,100) , 1 )
for (x,y,w,h) in right_eye:
r_eye=frame[y:y+h,x:x+w]
count=count+1
r_eye = cv2.cvtColor(r_eye,cv2.COLOR_BGR2GRAY)
r_eye = cv2.resize(r_eye,(24,24))
r_eye= r_eye/255
r_eye=  r_eye.reshape(24,24,-1)
r_eye = np.expand_dims(r_eye,axis=0)
rpred = model.predict_classes(r_eye)
if(rpred[0]==1):
lbl='Open'
if(rpred[0]==0):
lbl='Closed'
break
for (x,y,w,h) in left_eye:
l_eye=frame[y:y+h,x:x+w]
count=count+1
l_eye = cv2.cvtColor(l_eye,cv2.COLOR_BGR2GRAY)
l_eye = cv2.resize(l_eye,(24,24))
l_eye= l_eye/255
l_eye=l_eye.reshape(24,24,-1)
l_eye = np.expand_dims(l_eye,axis=0)
lpred = model.predict_classes(l_eye)
if(lpred[0]==1):
lbl='Open'
if(lpred[0]==0):
lbl='Closed'
break
```

```
if(rpred[0]==0 and lpred[0]==0):
score=score+1
cv2.putText(frame,"Closed",(10,height-20), font, 1,(255,255,255),1,cv2.LINE_AA)
# if(rpred[0]==1 or lpred[0]==1):
else:
score=score-1
cv2.putText(frame,"Open",(10,height-20), font, 1,(255,255,255),1,cv2.LINE_AA)
if(score<0):
score=0
cv2.putText(frame,'Score:'+str(score),(100,height-20), font, 1,(255,255,255),1,cv2.LINE_AA)
if(score>15):
#person is feeling sleepy so we beep the alarm
cv2.imwrite(os.path.join(path,'image.jpg'),frame)
try:
sound.play()
except:  # isplaying = False
pass
if(thicc<16):
thicc= thicc+2
else:
thicc=thicc-2
if(thicc<2):
thicc=2
cv2.rectangle(frame,(0,0),(width,height),(0,0,255),thicc)
cv2.imshow('frame',frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
break
cap.release()
cv2.destroyAllWindows()
```

#model program:

```
import os
from keras.preprocessing import image
import matplotlib.pyplot as plt
import numpy as np
from keras.utils.np_utils import to_categorical
import random,shutil

from keras.models import Sequential
from keras.layers import Dropout,Conv2D,Flatten,Dense, MaxPooling2D, BatchNormalization
from keras.models import load_model
def generator(dir, gen=image.ImageDataGenerator(rescale=1./255),
shuffle=True,batch_size=1,target_size=(24,24),class_mode='categorical' ):
return
gen.flow_from_directory(dir,batch_size=batch_size,shuffle=shuffle,color_mode='grayscale',class_mode
=class_mode,target_size=target_size)
BS= 32
TS=(24,24)
train_batch= generator('data/train',shuffle=True, batch_size=BS,target_size=TS)
valid_batch= generator('data/valid',shuffle=True, batch_size=BS,target_size=TS)
SPE= len(train_batch.classes)//BS
VS = len(valid_batch.classes)//BS
print(SPE,VS)
# img,labels= next(train_batch)
# print(img.shape)
model = Sequential([
Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=(24,24,1)),
MaxPooling2D(pool_size=(1,1)),
Conv2D(32,(3,3),activation='relu'),
MaxPooling2D(pool_size=(1,1)),
#32 convolution filters used each of size 3x3
#again
Conv2D(64, (3, 3), activation='relu'),
MaxPooling2D(pool_size=(1,1)),
```

```
#64 convolution filters used each of size 3x3
#choose the best features via pooling
#randomly turn neurons on and off to improve convergence
Dropout(0.25),
#flatten since too many dimensions, we only want a classification output
Flatten(),
#fully connected to get all relevant data
Dense(128, activation='relu'),
#one more dropout for convergence' sake :)
Dropout(0.5),
#output a softmax to squash the matrix into output probabilities
Dense(2, activation='softmax')
])
model.compile(optimizer='adam',loss='categorical_crossentropy',metrics=['accuracy'])
model.fit_generator(train_batch, validation_data=valid_batch,epochs=15,steps_per_epoch=SPE
,validation_steps=VS)
model.save('models/cnnCat2.h5', overwrite=True)
```

REFERENCES

- Angeline, P.J., Saunders, G.M., Pollack, J.B.: An evolutionary algorithm that constructs recurrent neural networks. IEEE Transactions on Neural Networks, 5(1), 54-65 (1994)
- Hinton, G.E., Zemel, R.S.: Autoencoders, minimum descriptionlength, and Helmholtz free energy. Advances in neuralinformation processing systems pp. 3-3 (1994)
- Al-Shawawreh, H., & Alsarhan, A. (2017). Real-time drowsiness detection using OpenCV. 2017 IEEE Jordan Conference on Applied Electrical Engineering and Computing Technologies (AEECT), Amman, Jordan, 1-5. doi: 10.1109/AEECT.2017.8257367
- Varghese, A., & Das, B. (2019). Real-time drowsiness detection using eye aspect ratio and OpenCV. 2019 International Conference on Robotics, Automation, Control and Embedded Systems (RACE), Kolkata, India, 1-5. doi: 10.1109/RACE48144.2019.898447
- Vuyyuru, B., & George, S. (2019). Real-time drowsiness detection system using OpenCV and Raspberry Pi. 2019 IEEE International Conference on Electrical, Computer and Communication Technologies (ICECCT), Coimbatore, India, 1-6. doi: 10.1109/ICECCT.2019.8869196
- Guo, Q., Liu, Z., & Zhu, G. (2018). Drowsiness detection based on OpenCV and RNN. 2018 IEEE 4th International Conference on Big Data Computing Service and Applications (BigDataService), Bamberg, Germany, 23-29. doi: 10.1109/BigDataService.2018.00040
- Saini, J., & Jain, A. (2021). Real-time drowsiness detection using OpenCV and machine learning techniques. 2021 International Conference on Computing, Communication and Signal Processing (ICCSP), Chennai, India, 1-6. doi: 10.1109/ICCSP52878.2021.9358045
- M. Suganya and H. Anandakumar, Handover based spectrum allocation in cognitive radionetworks, 2013 International Conference on Green Computing, Communication andConservation of Energy (ICGCE), Dec. 2013.doi:10.1109/icgce.2013.6823431.
- Haldorai and A. Ramu, An Intelligent-Based Wavelet Classifier for Accurate Prediction of Breast Cancer, Intelligent Multidimensional Data and Image Processing, pp. 306–319. Vural, E., Cetin,
- Vural, E., Cetin, M., Ercil, A., Littlewort, G., Barlett, M.: DrowsyDriver detection using facial movement analysis" Proc Springer-Verlag, Berlin; Heidelberg (2007)
- Vural, E., Bartlett, M.S., Littlewort, G., Cetin, M., Ercil, E.,Movellan, J.: Discrimination of moderate and acute drowsiness based on spontaneous facial expressions. IEEE International Conference on Pattern Recognition (2010)

- Q. Ji, Z. Zhu, P. Lan real-time nonintrusive monitoring andprediction of driver fatigue. IEEE Transactions on vehiculartechnology 53(4) (2004)
- Chen, H., Murray, A.F.: Continuous restricted Boltzmann machine with an implementable training algorithm. IEEE Proceedings - Vision, Image and Signal Processing 150, (3), 153-158., (2003)
- Salakhutdinov, R., Mnih, A., Hinton, G.: Restricted Boltzmann machines for collaborative filtering. Proc. of the 24th international conference on Machine learning pp. 791-798 (2007)
- Singh, M., Kaur, G.: Drowsy detection on eye blink durationusing algorithm. International Journal of Emerging Technology and Advanced Engineering 2(4) (2012)